WasteWatch — Full System Flow

Container → module → function walkthrough, plus planned MiniCPM-o narration addition

1. Containers (docker-compose)

Container	Job
mediamtx	Camera ingest gateway — RTSP/RTMP/HTTPS in, normalizes to RTSP
pipeline	DeepStream GPU process — detect, track, FSM, evidence capture, triggers VLM
kafka	Message bus between everything downstream of pipeline
vlm	Model server hosting the VLM — Qwen2.5-VL today (mainline vLLM); MiniCPM-o 4.5 planned via **vllm-omni** (separate omni-modal extension of vLLM, not mainline)
vlm-worker	gRPC service wrapping vlm — receives Analyze calls from pipeline
api	FastAPI backend — REST/WebSocket, DB access, dashboard/history browser
postgres	Persistent store — cameras, violation events, evidence, alerts
redis	Cache/session state
alert-svc	Consumes alerts, fans out to Telegram/SIP/speaker/WebSocket
sip-svc	Phone call escalation
tts / tts-worker	Text-to-speech for speaker announcements
lpr-worker	License plate recognition (PaddleOCR)
evidence-svc	Records clips/snapshots, syncs to Dropbox
log-worker	Structured logs → Qdrant (searchable)
health-monitor / healer	Watches camera/service health, auto-recovers
reports	Scheduled report generation
keycloak	Auth
qdrant	Vector DB for log search

2. Frame-level flow `inside pipeline container`

```
Camera (RTSP/RTMP/HTTPS)
   ↓
MediaMTX (normalizes → RTSP)
   ↓
DeepStream GStreamer pipeline:
   streammux (batches multi-camera frames)
     ↓
   nvinfer — RF-DETR detector (TensorRT) → boxes + class + confidence
     ↓
   nvtracker — ByteTrack → assigns track_id, tracks object across frames
     ↓
   nvdsanalytics — point-in-polygon ROI check → drop anything outside zone
     ↓
   [wastewatchmeta / FSM update] — event_fsm.py: per-track state machine
     ↓
   nvosd — draws boxes/labels for output video
     ↓
   tee → 3 parallel branches:
```

```
    ├─ Branch 1: nvstreamdemux → per-camera RTSP out → MediaMTX → frontend viewer
    ├─ Branch 2: fakesink_meta → extract_metadata() probe → Kafka (wastewatch.metadata/events)
    └─ Branch 3: fakesink_capture → capture_frame_probe() → JPEG cache per camera
```

3. FSM per tracked object `EventFSMManager.update()`

```
IDLE → (enters ROI) → ENTER → (dwell > 5s) → OBSERVE
                                                ↓ (dwell > 15s OR suspicious motion)
                                            VIOLATION
                                                ↓
                                            ALERTING → ESCALATED (if repeat/ignored)
   (leaves ROI early) → EXITED
```

On ENTER/VIOLATION, extract_metadata() calls vlm_client.submit(...) (gRPC, fire-and-forget) with the cached JPEG crop.

4. VLM trigger path — function chain

```
pipeline.py: extract_metadata()
   → vlm_grpc_client.py: VLMGrpcClient.submit()  [fire-and-forget, thread pool]
      → gRPC Analyze RPC
         → grpc_server.py: VLMServicer.Analyze()
            → client.py: VLMClient.analyze()
               → _call_vlm() → HTTP POST to vllm server /v1/chat/completions
            ← parses JSON response (is_dumping, severity, waste_type, deterrent_message)
         → publishes result to Kafka wastewatch.vlm.results
   ← pipeline thread never blocks, moves on to next frame
```

5. Downstream from Kafka `api / worker containers`

```
wastewatch.vlm.results → alert dispatcher → decides severity
                                          → Kafka wastewatch.alerts
                                                ↓
                        ┌──────────┬──────────┬──────────┐
                        ▼          ▼          ▼          ▼
                   Telegram   WebSocket   SIP call   IP Speaker
                   (bot msg)  (live dashboard) (phone)  (TTS via wastewatch.tts)

wastewatch.events (FSM state change) → evidence-svc:
   VIOLATION → start recording (pre+post buffer clip)
   EXITED    → stop recording → save → sync to Dropbox → evidence_writer JSON sidecar

wastewatch.lpr → lpr-worker → PaddleOCR → plate_reads table
```

6. History Browser `API read path`

```
Frontend TimelineView
   → GET /api/evidence/files
      → evidence.py: list_evidence_files()
         → scans EVIDENCE_ROOT filesystem for evt_*.json sidecars
         → resolves camera name → DB UUID
         → returns newest-first list
   → GET /api/evidence/file?path=...  → serves the actual clip/snapshot
```

7. Planned addition — narration

from spec/plan, not yet live

```
capture_frame_probe (already runs) → frame_jpeg_cache
                                          ↓
                          GLib timer @ WW_NARRATION_FPS → narration_tick()
                                          ↓
                          NarrationClient.submit_frame() [MiniCPM-o duplex session]
                                          ↓
                          flags found? → Kafka wastewatch.narration.flags
                                          ↓
                          NarrationFlagWorker → narration_flags table (review_status=pending)
                                          ↓
                          Admin reviews via API → confirm → promotes to ViolationEvent
                                                → dismiss → closed, no event created
```

Today's plan ships NarrationClient with a stubbed `_narrate()` — the real MiniCPM-o backend below is the follow-up swap.

8. MiniCPM-o 4.5 model internals

what NarrationClient._narrate() swaps to

Property	Value
Params	9B (SigLip2 vision encoder + Whisper-medium audio encoder + CosyVoice2 speech decoder + Qwen3-8B backbone)
VRAM	BF16 ≈19GB · int4 ≈11GB (AWQ/GPTQ not confirmed available for 4.5)
Serving	vllm-omni (not mainline vLLM) — exposes /v1/chat/completions AND a stateful duplex streaming API
Trigger-path mode	Turn-based, OpenAI-style chat completions, init_audio=False, enable_thinking=False (same shape as Qwen2.5-VL today — Phase 1 swap is config-only)
Narration-path mode	Native "Omni-Flow" duplex streaming — NOT the same call as the trigger path

```
Real-time narration internals (Omni-Flow framework):

  Video frames ─┐
                ├─→ shared 1-second chunk timeline ─→ TAIL alignment
  Audio (opt)  ─┘    (Time-Aligned Interleaving — keeps speech
                       in sync with video, doesn't drift)

  Per chunk:
    video → compressed to 64 vision tokens  (16x compression)
    audio → 50 tok/s compressed to 10 tok/s (5x compression)  [unused, init_audio=False]

  Session API (stateful, one long-lived session per camera):
    streaming_prefill(chunk)   → feed one 1s chunk into running context
    streaming_generate()       → pull narration/flag output for that chunk
                                 (NOT a fresh request each time — same
                                 session accumulates context, unlike the
                                 stateless trigger-path chat completion)

  Why 1s chunks: paper found 0.1–0.2s chunks starved the model of context
  and degraded narration quality — 1s is the validated minimum window.
```

Risk flagged in the design spec: vllm-omni is less mature than mainline vLLM's Qwen2.5-VL support (fewer users, less version-pinning history) — treated as an operational risk to watch, not a blocker.